FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de Tratamiento de Datos

Bases de Datos No Relacionales (NoSQL)

Año: 2025

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Contenido

¿Por qué surgió NoSQL?. Un poco de historia 3
LIMITACIONES DE LAS BASES RELACIONALES 3
Bases de datos diseñadas para la web moderna 4
SQL vs NoSQL: Tabla Comparativa 5
CLASIFICACIÓN NoSQL 6
Clave-Valor 6
Motores: Redis, Amazon DynamoDB, Azure Cosmos DB (modo KV), Riak. Concepto: Cada dato es un par único clave → valor (string, JSON, blob, etc.). Ventajas: Acceso en O(1), estructura super-simple, ideal para escalar. Casos de uso típicos: 6
Documentales 6
Ejemplos y casos de uso – Redis, MongoDB 7
Modelado de datos documental (MongoDB)....8
Consultas básicas (shell MongoDB) 8
Columnares (Wide-Column)....8
Grafo 9
Ejemplos y casos de uso - Apache Cassandra, Hbase, Neo4j, Amazon Neptune 9
Ejemplo Cassandra (CQL)....10
Grafo (ej. Neo4j, Amazon Neptune)....10
El teorema CAP 11
¿Qué es el teorema CAP?....11
Los tres tipos de decisiones posibles....13
Consistencia Eventual vs. ACID 14
Consistencia Eventual (BASE)....15
Data Warehousing y OLAP....16
Introducción....16
Componentes clave 16
Esquemas de modelado 16
OLAP (On-Line Analytical Processing) 17
Resumen práctico 17

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

BASES DE DATOS NoSQL: De lo Relacional a lo Distribuido

¿Por qué surgió NoSQL?. Un poco de historia

A mediados de los 90 la web era principalmente estática y los usuarios contados por miles; los modelos relacionales bastaban. El salto llegó con el auge de Google, Facebook, Amazon y Twitter (2004-2008): millones de usuarios simultáneos, peticiones globales y datos que ya no eran solo "cuentas bancarias" sino logs, geolocalizaciones, fotos, likes y sensores. En 2007 Amazon publica el paper de Dynamo y en 2009 se lanza MongoDB; el término "NoSQL" se acuña en el meetup de San Francisco de ese mismo año para describir bases que rompían con el modelo relacional tradicional.

LIMITACIONES DE LAS BASES RELACIONALES

- Escalabilidad vertical costosa: para soportar más carga hay que comprar servidores más grandes (más CPU, RAM, discos), lo que se vuelve económicamente insostenible cuando los usuarios crecen de cientos a millones.
- Esquemas rígidos y migraciones complejas: cualquier cambio en una tabla (añadir un campo, alterar tipos de datos) requiere bloqueos, scripts de migración y pruebas que ralentizan el desarrollo ágil.
- Dificultad para manejar datos no estructurados: XML, JSON, logs, imágenes, eventos de IoT o redes sociales no encajan bien en filas y columnas fijas, obligando a normalizar excesivamente o a guardar blobs opacos.
- Rendimiento limitado en operaciones distribuidas: el cumplimiento estricto de ACID implica bloqueos y coordinación

entre nodos, lo que eleva la latencia cuando la base está repartida en varios continentes.

- Alta complejidad cuando el modelo de datos cambia con frecuencia: start-ups, SaaS y aplicaciones móviles iteran semanalmente; esperar a que el DBA modifique el esquema y valide millones de registros rompe la velocidad de negocio.

Bases de datos diseñadas para la web moderna

Internet pasó de tener miles de usuarios simultáneos a millones en minutos; los datos dejaron de ser solo pedidos y facturas para convertirse en clicks, geolocalizaciones, likes y sensores. NoSQL nació para:

- Escalar horizontalmente (añadir servidores baratos en vez de "servidores más grandes").
- Aceptar esquemas flexibles o incluso ausentes, permitiendo que cada registro evolucione sin parar la aplicación.
- Ofrecer alta disponibilidad y latencia de un dígito de milisegundos a escala global, aunque eso implique consistencia eventual.

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

SQL vs NoSQL: Tabla Comparativa

CARACTERÍSTICA	SQL (RELACIONAL)	NoSQL (NO RELACIONAL)
Esquema	Fijo: tablas, columnas y tipos definidos antes de insertar el primer dato.	Flexible: cada registro puede tener campos distintos; se añaden "sobre la marcha" sin migraciones.
Escalabilidad	Vertical: se compra un servidor más grande (más CPU, RAM, discos).	Horizontal: se añaden servidores baratos; el clúster crece sin parar la aplicación.
Transacciones	ACID: Atomicidad, Consistencia, Aislamiento, Durabilidad → datos 100 % seguros.	BASE: Basically Available, Soft state, Eventual consistency → prioriza disponibilidad y velocidad.
Consultas	Lenguaje SQL estándar: SELECT … JOIN … WHERE idéntico en MySQL, PostgreSQL, Oracle.	APIs propias: JSON sobre HTTP, métodos específicos por motor (MongoDB: db.collection.find(), Cassandra: CQL, etc.).
Consistencia	Fuerte: todos los usuarios ven el mismo valor en cualquier instante.	Eventual: en pocos ms/s los nodos se sincronizan; durante ese lapso pueden existir lecturas distintas.

CARACTERÍSTICA	SQL (RELACIONAL)	NoSQL (NO RELACIONAL)
Estructura	Tablas con filas y columnas; relaciones mediante claves foráneas.	Variada: documentos (JSON/BSON), pares clave-valor, column families o grafos según el caso de uso.

CLASIFICACIÓN NoSQL

Las bases de datos NoSQL se clasifican en cuatro modelos fundamentales —clave-valor, documental, columnar y grafo— según la estructura lógica que utilizan para organizar y relacionar los datos, diferenciados por la forma en que almacenan la información y el tipo de operaciones que optimizan.

Clave-Valor

Motores: Redis, Amazon DynamoDB, Azure Cosmos DB (modo KV), Riak.
Concepto: Cada dato es un par único clave → valor (string, JSON, blob, etc.).
Ventajas: Acceso en O(1), estructura super-simple, ideal para escalar.
Casos de uso típicos:

- Caché de consultas o páginas (evita golpear la base lenta).
- Almacenamiento de sesiones de usuario (cookie <-> token).
- Colas de publicación-suscripción con TTL.

Documentales

Motores: MongoDB, Couchbase, RavenDB, Amazon DocumentDB.
Concepto: Los datos se guardan como documentos (JSON, BSON, XML)

dentro de “colecciones”; cada documento puede tener campos distintos y anidarse.
Ventajas: Esquema dinámico, indexación flexible, consultas ricas sin JOINs.
Casos de uso típicos:

- Perfiles de usuario (datos personales, preferencias, listas internas).
- Catálogos de e-commerce (productos con atributos variables: tallas, colores, peso).
- CMS y blogs (entradas, metadatos, tags, comentarios embebidos).

Ejemplos y casos de uso – Redis, MongoDB

Modelado de datos en clave-valor (Redis)

Redis almacena cada dato como un par clave → valor, donde el valor puede ser string, hash, lista, conjunto o sorted-set. Esto permite representar estructuras simples sin esquema previo.

Casos típicos:

- Carrito de compras: la clave “user:1234:cart” contiene un JSON-array con sku y cantidad.
- Contador de likes: la clave “post:987:likes” se incrementa atómicamente.

Consultas básicas (CLI Redis)

```
SET user:1234:cart '[{"sku":"A1","qty":2}]'
OK
GET user:1234:cart
"[{"sku":"A1","qty":2}]"
```

INCR post:987:likes
(integer) 1

Modelado de datos documental (MongoDB)

Los documentos se guardan en formato BSON (JSON binario) dentro de colecciones; cada documento puede tener campos distintos y anidados sin alterar el resto.
Ejemplo: colección “usuarios”

```
{ "_id": 1234,
"nombre": "Ana",
"email": "ana@mail.com",
"direccion": {"calle": "Serrano", "cp": 28001},
"carrito": [{"sku": "A1", "qty": 2, "precio": 15.5}]
}
```

Consultas básicas (shell MongoDB)

```
db.usuarios.find({"nombre": "Ana"})
db.usuarios.updateOne(
{_id: 1234},
{$push: {carrito: {sku: "B7", qty: 1, precio: 9.9}}}
)
```

Columnares (Wide-Column)

Motores: Apache Cassandra, DataStax, HBase, ScyllaDB.
Concepto: Filas identificadas por clave, pero cada fila puede contener millones de columnas agrupadas en “familias”; se escribe y lee por

rangos de columnas.
Ventajas: Escritura masiva rápida, compresión, alta disponibilidad sin punto único de fallo.
Casos de uso típicos:

- Series temporales de sensores IoT (timestamp → valor).
- Analytics de click-stream (millones de eventos/segundo).
- Contadores distribuidos (vistas, "me gusta", tiempo de uso).

Grafo

Motores: Neo4j, Amazon Neptune, ArangoDB (modo grafo), TigerGraph.
Concepto: Nodos (entidades) + Relaciones (aristas) + Propiedades en ambos; todo indexado para navegación profunda sin JOINs costosos.
Ventajas: Recorridos complejos en tiempo constante por arista, independiente del tamaño total.
Casos de uso típicos:

- Redes sociales (amigos → amigos de amigos, distancia 3-4 saltos).
- Sistemas de recomendación ("clientes que compraron X también compraron Y").
- Detección de fraude (anillos de transacciones, patrones de lavado).
- Gestión de derechos y accesos (RBAC, ACL con herencia).

Ejemplos y casos de uso - Apache Cassandra, Hbase, Neo4j, Amazon Neptune

Columnares (ej. Apache Cassandra, HBase)

Datos organizados por clave de fila y familias de columnas; optimizados para inserciones masivas y lecturas por rango de clave.

Ejemplo Cassandra (CQL)

```
-- Crear keyspace y tabla de series temporales

CREATE KEYSPACE iot WITH replication = {'class':'SimpleStrategy',
'replication_factor':3};
USE iot;
CREATE TABLE temps (
sensor_id text,
ts timestamp,
valor double,
PRIMARY KEY (sensor_id, ts)
) WITH CLUSTERING ORDER BY (ts DESC);

-- Insertar y consultar

INSERT INTO temps (sensor_id, ts, valor)VALUES ('S1',
toTimestamp(now()), 23.7);
SELECT * FROM temps WHERE sensor_id='S1' AND ts > '2025-06-01'
LIMIT 100;
```

Casos: métricas de IoT, logs de click-stream, contadores distribuidos.
Ventaja: escalado horizontal sin punto único de fallo y compresión eficiente por columna.

Grafo (ej. Neo4j, Amazon Neptune)

Modelo nodos + relaciones + propiedades; los recorridos profundos cuestan O(aristas) y no O(tablas).

Ejemplo Neo4j (Cypher)

```
-- Crear nodos y relación
```

```
CREATE (a:Persona {nombre:'Ana'})
CREATE (b:Persona {nombre:'Luis'})
CREATE (a)-[:AMIGO]->(b);

-- Amigos de amigos (distancia 2)

MATCH (ana:Persona {nombre:'Ana'})-[:AMIGO*2..2]->(fof)
RETURN DISTINCT fof.nombre;
```

Casos: redes sociales (“amigos de amigos”), recomendaciones, detección de fraude.
Ventaja: consultas complejas de relaciones (caminos, centrality) sin costosos JOINs.

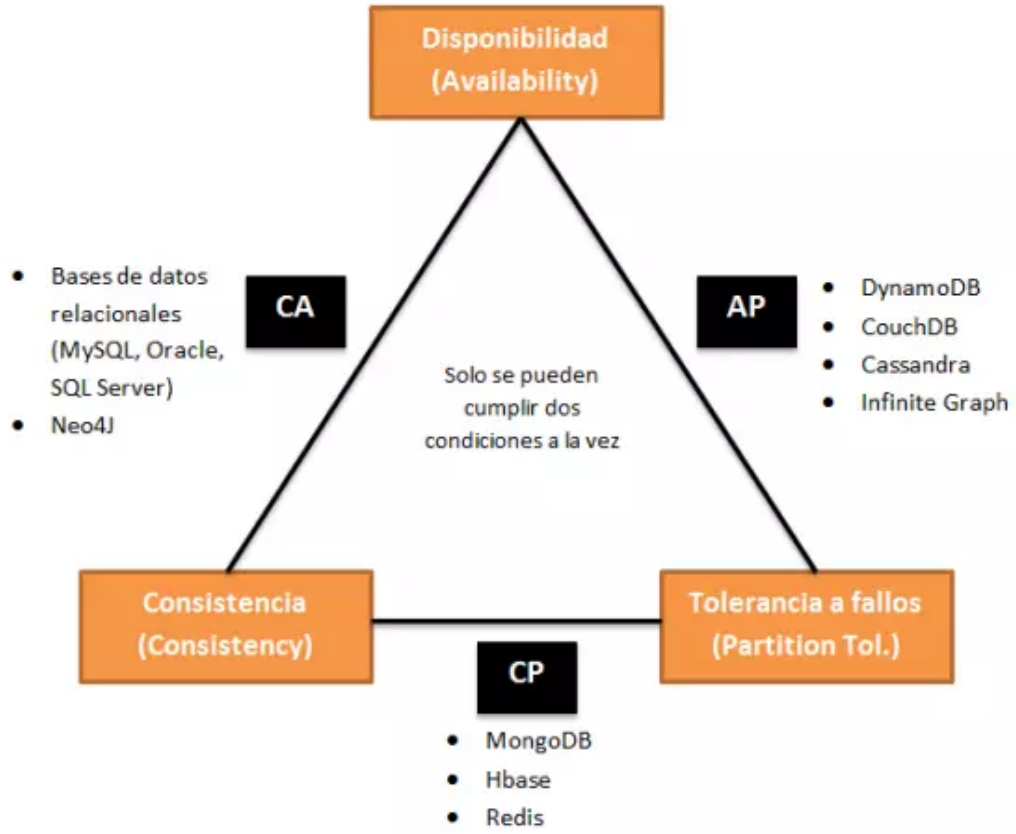
El teorema CAP

¿Qué es el teorema CAP?

El teorema CAP dice que, en un sistema distribuido (o sea, con varios nodos conectados por red), cuando ocurre un problema de comunicación entre ellos, solo podés garantizar dos de estas tres propiedades:

Letra	Significa	Qué implica
C – Consistency (Consistencia)	Todos los nodos ven el mismo dato al mismo tiempo.	Si actualizás algo en un nodo, cualquier otro que lo lea obtiene **exactamente el mismo valor**.

Letra	Significa	Qué implica
A – Availability (Disponibilidad)	El sistema siempre responde, incluso si no puede garantizar que el dato esté actualizado.	Aunque un nodo falle o la red se corte, el sistema **no se detiene**.
P – Partition Tolerance (Tolerancia a particiones)	El sistema sigue funcionando aunque haya fallas o cortes en la red.	Si los nodos no pueden comunicarse, el sistema igual acepta peticiones.



Los Sistemas Distribuidos solo pueden garantizar 2 de 3:

- CA sin P: imposible en la práctica; las redes fallan.
- CP sin A: bancos, contabilidad (consistencia + partición).
- AP sin C: redes sociales, carritos de compra (disponibilidad + partición) aceptan consistencia eventual.

Los tres tipos de decisiones posibles

CA – Consistente y Disponible (sin partición)

Funciona cuando la red está sana y todos los nodos pueden comunicarse.
Ejemplo:
Un MySQL centralizado en un solo servidor o en una LAN controlada.

- Cada lectura ve el último dato (C).
- Siempre hay respuesta (A).
  Pero… si se corta la red, deja de estar disponible no tolera particiones.

CP – Consistente y Tolerante a particiones

Prioriza **la exactitud de los datos** sobre la disponibilidad.
Ejemplo:
MongoDB o HBase en modo fuerte.
Si el nodo principal no puede confirmar que los demás recibieron el cambio, bloquea las escrituras hasta restablecer la comunicación.
Los datos son correctos, pero algunos usuarios tendrán errores de "no disponible".
Analogía:
Un cajero automático que no te deja extraer si perdió conexión con el banco — prefiere no operar antes que darte un saldo incorrecto.

AP – Disponible y Tolerante a particiones

Prioriza seguir respondiendo, aunque sea con datos viejos (consistencia eventual).
Ejemplo:
Cassandra, DynamoDB, Couchbase.
Si un nodo no puede comunicarse, igual acepta lecturas y escrituras.
Después, cuando la red se repara, sincroniza las diferencias (reconciliación).
Siempre responde.
Pero puede devolver datos "stale" temporalmente.

Analogía:
Un local que sigue vendiendo aunque el sistema central esté caído; anota las ventas y las sube después.

Consistencia Eventual vs. ACID

¿Qué es ACID?

Conjunto de propiedades que garantiza que cada transacción de una base relacional es segura, incluso en presencia de fallos:

A – Atomicidad

Todas las operaciones dentro de la transacción se ejecutan completamente o ninguna.
Ejemplo: en la transferencia bancaria SQL, si la segunda UPDATE falla, el motor deshace el primer descuento y el saldo vuelve a 100.

C – Consistencia

Al finalizar la transacción el sistema pasa de un estado válido a otro estado válido (reglas, triggers, claves foráneas).

I – Aislamiento

Transacciones concurrentes no se "ven" entre sí; produce el mismo resultado que si se hubieran ejecutado en serie.

D – Durabilidad

Una vez el COMMIT es confirmado, los cambios persisten aunque venga un apagón.

Consistencia Eventual (BASE)

En las bases NoSQL distribuidas, cumplir las cuatro al pie de la letra es difícil,
porque se prioriza velocidad y disponibilidad sobre consistencia inmediata.
Por eso, muchos motores NoSQL se rigen por un principio alternativo llamado BASE:

- Basically Available (básicamente disponible)
- Soft-state (el estado puede cambiar con el tiempo)
- Eventually consistent (eventualmente consistente)

Es decir: no garantizan ACID perfecto, pero a cambio responden rápido y escalan mejor.

El sistema acepta que, tras una escritura, diferentes réplicas pueden mostrar valores distintos durante un intervalo de tiempo (ms, s). Sin embargo, garantiza que, pasado ese tiempo, todos los nodos convergen

al mismo valor.
Ventaja: disponibilidad y latencia baja; coste: breve “vista borrosa” de los datos.

Data Warehousing y OLAP

Introducción

El Data Warehouse (DW) es un repositorio centralizado, orientado a temas de negocio, integrado, no volátil y variable en el tiempo, diseñado para soportar la toma de decisiones mediante análisis históricos en lugar de operaciones transaccionales cotidianas.

Componentes clave

- ETL (Extract-Transform-Load): proceso que limpia, homologa y carga datos desde sistemas operacionales al DW.
- Almacén de datos propiamente dicho: esquemas en estrella o copo de nieve con tablas de hechos y dimensiones.
- Metadatos: información sobre origen, transformaciones y significado de cada campo.
- Herramientas de acceso: OLAP, Data Mining, BI, dashboards.

Esquemas de modelado

Estrella: tabla de hechos central (ventas, inventario) rodeada de tablas de dimensión (tiempo, cliente, producto).

Copo de nieve: dimensión normalizada en varias tablas (p. ej. cliente → ciudad → país).

OLAP (On-Line Analytical Processing)

Motor que permite navegar por los datos agregados con operaciones "slice and dice", drill-down/up, roll-up y pivotado.
Tipos:
- MOLAP: cubos multidimensionales precalculados (rápidos, ocupan más espacio).
- ROLAP: consultas SQL sobre el DW relacional (sin precálculo, más flexibles).
- HOLAP: híbrido; hechos recientes en ROLAP, agregados históricos en MOLAP.

Cuándo usar un DW/OLAP hoy

- Reportes gerenciales que combinan fuentes heterogéneas y requieren historia.
- Análisis de tendencias, pronósticos, segmentación de clientes.
- Aunque los data lakes modernos (Hadoop, Snowflake, BigQuery) han absorbido gran parte del volumen, el DW sigue siendo la capa "curated" que alimenta dashboards corporativos y cumplimiento normativo.

Resumen práctico

ETL -→ DW (estrella/copo) -→ Cubo OLAP -→ Dashboard.
Proporciona consistencia histórica y rendimiento agregado, a cambio de mayor latencia de carga y esquemas más rígidos que los data lakes.